Week 4: Fourier Transform

Overview

Class meeting: 03 Feb 2026

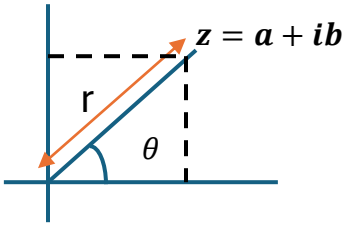
- complex numbers
  1. **standard form**
  2. polar form
  3. **view as rotation and scale matrices**
- Discrete Fourier Transform (DFT)
- **Fourier Series**
- use numerical libraries to compute DFT/frequency information encoding
- **view DFT as product of an orthogonal matrix**
- use DFT to analyze a signal where the frequency varies in time

Async

Fourier Transform

Complex Numbers (CNs)	• **complex numbers** (C) enable capturing roots of certain polynomials
	  ○ $0 = x^2 + 1$
	  ○ $0 = x^2 + 2x + 2 = (x+1)^2 + 1$
	• **Add** an **imaginary** number ***i*** with $i\text{^}2 = -1$
	  ○ $0 = x^2 + 1 => x = \pm i$
	  ○ $0 = x^2 + 2x + 2 \ => (x+1)^2 + 1 = 0 => x = -1 \pm i$

CNs as Matrix	• $1 <\rightarrow \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ • $i <\rightarrow \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, i^2 = -1$, counterclockwise $\mathbf{90^0}$ **rotation**
CNs Standard Format	• $z = a + bi, where\ a\ is\ the\ real\ part\ and\ ib\ is\ the\ imaginary\ part$ • $a + ib = a\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + b\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} a & -b \\ b & a \end{bmatrix}$

CNs Polar Format	• $r = \|z\| = \sqrt{a^2 + b^2}$ • $a + ib = r\cos\theta + i\sin\theta$ • $a + ib = r\begin{bmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{bmatrix}$ 
CN Multiplication	• **1** *and* ***i*** **commute** so general **multiplications** work **as expected** ○ $(a+ib)(c+id) = ac + ibc + iad + i\text{^}2\ bd$ $= ac - bd + i(bc + ad)$ ○ $(a+ib)(c+id) \rightarrow \begin{bmatrix}a & -b \\ b & a\end{bmatrix}\begin{bmatrix}c & -d \\ d & c\end{bmatrix} =$ $\begin{bmatrix}ac - bd & -ad - bc \\ ad + bc & ac - bd\end{bmatrix}$ • **exponential** form ○ $e^{i\theta} = cos\theta + i\ sin\theta$ ○ $e^{i\alpha + i\beta} = e^{i\alpha}e^{i\beta}$ ○ $z = a + ib = r\,e^{i\theta}$ (polar form; magnitude; direction: $e^{i\theta}$)
Complex Conjugate	• a **complex conjugate** (**CC**) is formed by **reversing** the **sign** of the **imaginary** part of a complex **number** • $\overline{x + \imath y} = x - iy$ ← **mirroring** along the **x-axis** (*complex conjugate*) • $\bar{z}z = (x + iy)(x - iy) = x^2 + y^2 = \|z\|^2$ ← CN*CC = $\mathbb{R}$ ($magnitude$)^2 CC: https://www.khanacademy.org/math/
Real and Imaginary Parts	• $re(z) = \frac{z+\bar{z}}{2}$ $\quad$ $\mathrm{im}(z) = \frac{z+\bar{z}}{2i} = -i\frac{z+\bar{z}}{2}$
Hermitian Matrix	• A **Hermitian** matrix is a **square complex matrix A** equal to its own conjugate transpose and equates to the **conjugate transpose** of the **matrix / vector** ○ $A* = A^H = \overline{(A^T)}$ Hermitian: https://en.wikipedia.org/wiki/Hermitian_matrix

Inner Product

- $u \cdot v = u^H v = \sum_i \overline{u_i}\, v_i$
- $u \cdot v = \overline{v \cdot u}$

CN and Imaginary Numbers in Julia

```
julia> z1 = 2 + 3im
2 + 3im

julia> z2 = 1 - 1im
1 - 1im

julia> norm(z1)
3.605551275463989

julia> norm(z2)
1.4142135623730951

julia> z1*z2
5 + 1im

julia> z1/z2
-0.5 + 2.5im

julia> exp(1 + im)
1.4686939399158851 + 2.2873552871788423im

julia> exp(1)*cos(1)
1.4686939399158851

julia> sqrt(complex(-1))
0.0 + 1.0im

julia> z = 3 + 2im
3 + 2im

julia> sqrt(z)
1.8173540210239707 + 0.5502505227003375im

julia> conj(z)
3 - 2im

julia> real(z)
3

julia> imag(z)
2

julia> angle(z)
0.5880026035475675
```

'*im*' declares the imaginary part of the CN

get modulus

CN multiplication

you get it...

conversion → complex

Fourier Transform

- A **Fourier** transform (FT) is a mathemetical technique that **transforms** a function of **time** $x(t)$, to a fuciton of **frequency** $X(\omega)$
- **Three** forms
  - Fourier **transform**
  - Fourier **series**
  - **discrete** Fourier transform (**DFT**)

"Fourier Transform" colloquially and in this class

- **Fourier** transform ↔ **inverse** transform
  - **FTs** are **invertible** using **inverse** of FT **matrix** F^{-1}
- For analytical functions on $\mathbb{R}$
  - $F(\omega) = \int_{-\infty}^{\infty} f(t)e^{-i\omega t}\,dt$
  - $f(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} F(\omega)e^{-i\omega t}\,d\omega$
  - **Euler's** formula: $e^{i\omega t} = \cos(\omega t) + i\sin(\omega t)$
    - where e is the **base** of the **natural log**arithm, i is the imaginary unit $(\mathbf{i^2 = -1})$, and x is the **angle** in **radians**
    - **bridges** complex **analysis** and trigonometry by relating the **exponential** function to **sine** and **cosine**

Fourier Transform: https://lpsa.swarthmore.edu/Fourier/Xforms/FXformIntro.html

Euler's Formula: https://www.youtube.com/watch?v=CRj-sbi2i2I

Fourier Series	• for anlytical **functions** that are **periodic** • **a series expansion of a periodic function into a sum of trigonometric functions** ○ $c_n = \frac{1}{L}\int_0^L f(x)\, e^{\frac{i\,2\pi n\,x}{L}}, dx$ ○ $f(x) = \sum_{-\infty}^{\infty} c_n e^{\frac{i\,2\pi n\,x}{L}}$ • **we will not be using this in this class** FS: https://en.wikipedia.org/wiki/Fourier_series
Discrete Fourier Transform	• a **fundamental** mathematical **technique** in **digital** signal **processing** that **converts** a **finite** sequence of evenly spaced data **samples**, typically representing a **signal** in the **time** or **spatial** domain, into its corresponding **frequency** domain **representation**. It computes the **amplitude** and **phase** of different **frequencies** present in the signal, enabling **analysis** and **manipulation** of, for instance, audio or image data. $$c_k = \frac{1}{N}\sum_{n=0}^{N-1} x_n \exp(-\mathrm{i}\frac{2\pi i\, k\, n}{N})$$ $$z_n = \sum_{k=0}^{N-1} c_k \exp\left(-\mathrm{i}\frac{2\pi i\, k\, n}{N}\right)$$ DFT: https://en.wikipedia.org/wiki/Discrete_Fourier_transform

sin Transform

- **a** less complicated **approach is to take back to a** simpler **transform, the** sin **transform is related to the** DFT

$$v_k^i = sin\left(\pi k \frac{i}{n}\right), k = 1, \dots, n-1, \qquad i = 1, \dots, n-1$$

$n-1$ vectors, each with $n-1$ values

- **all of those** vectors **are** orthogonal **to eachother**

$$v_i \cdot v_j = 0, if\ k \neq j$$

$$\boldsymbol{v_i \cdot v_j = \sum_{i=1}^{n-1} sin\left(\pi k \frac{i}{n}\right) sin\left(\pi k \frac{j}{n}\right) = 0}$$

  - $n-1$ orthogonal **values with** $n-1$ entries**, span all of** $\mathbb{R}^{n-1}$**, so they form am** orthogonal basis

- **still, they are** not **quite** orthonormal **(orthogonal unit vectors)**

$$\textbf{since, } ||v_k||^2 = v_k \cdot\ v_k = \sum_{i=1}^{n-1} \sin^2\left(\pi k \frac{i}{n}\right)^2 = \frac{n}{2}$$

- **create a** basis matrix
  - $V = [v_1, v_2, \cdots, v_{n-1}]$

    $V^T V = \frac{n}{2} I \quad \rightarrow \quad V^{-1} = \frac{2}{n} V^T$

    reminder: $\boldsymbol{V^T = V^{-1}}$*, if* $\boldsymbol{V}$ *is an* ***orthogonal matrix***
  - **ex: for a** $basis\ V$**, there is a** $coordinate\ c$ **such that there is a** vector x **that** $spans\ V$**, such that:** $x = Vc$
  - **any** vector **with** $n-1$ **entries can be** spanned **by the** $V\ vectors$

    reminder: a ***basis*** *is a set of* ***vectors*** *that acts a* ***foundational, minimal building block*** *for a* ***vector space***

$$x = Vc \Rightarrow V^T x = V^T V c = \frac{n}{2} c \Rightarrow c = \frac{2}{n} V^T x$$

$$c_k = \frac{2}{n} v_k \cdot x, k = 1, \dots, n-1$$

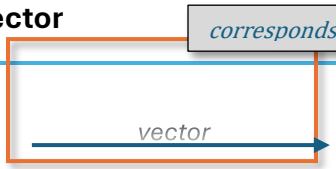
- ***putting this together*:** start **with a** $vector\ x$

$$\boldsymbol{x = Vc = c_1 v_1 + \cdots + c_{n-1} v_{n-1} \Rightarrow\ x_i = \sum_{k=1}^{n-1} c_k\ sin(\pi k \frac{i}{n})}$$

- where the $\boldsymbol{c\ values}$ come from the **vector**

Decompose a Signal

corresponds to: $|v_k|_i$



$$c = \frac{2}{n} V^T x \Rightarrow c_k = \frac{2}{n} v_k \cdot x \Rightarrow c_k = \frac{2}{n} \sum_{i=1}^{n-1} x_i \, sin(\pi k \frac{i}{n})$$

- **compare** to the **DFT** equation

$$c_k = \frac{1}{N} \sum_{n=0}^{N-1} x_n \exp\left(-i \frac{2\pi k n}{N}\right) \qquad z_n = \sum_{k=0}^{N-1} c_k \exp\left(i \frac{2\pi k n}{N}\right)$$

vector decomposition

- any **vector** can be **decomposed** like this:

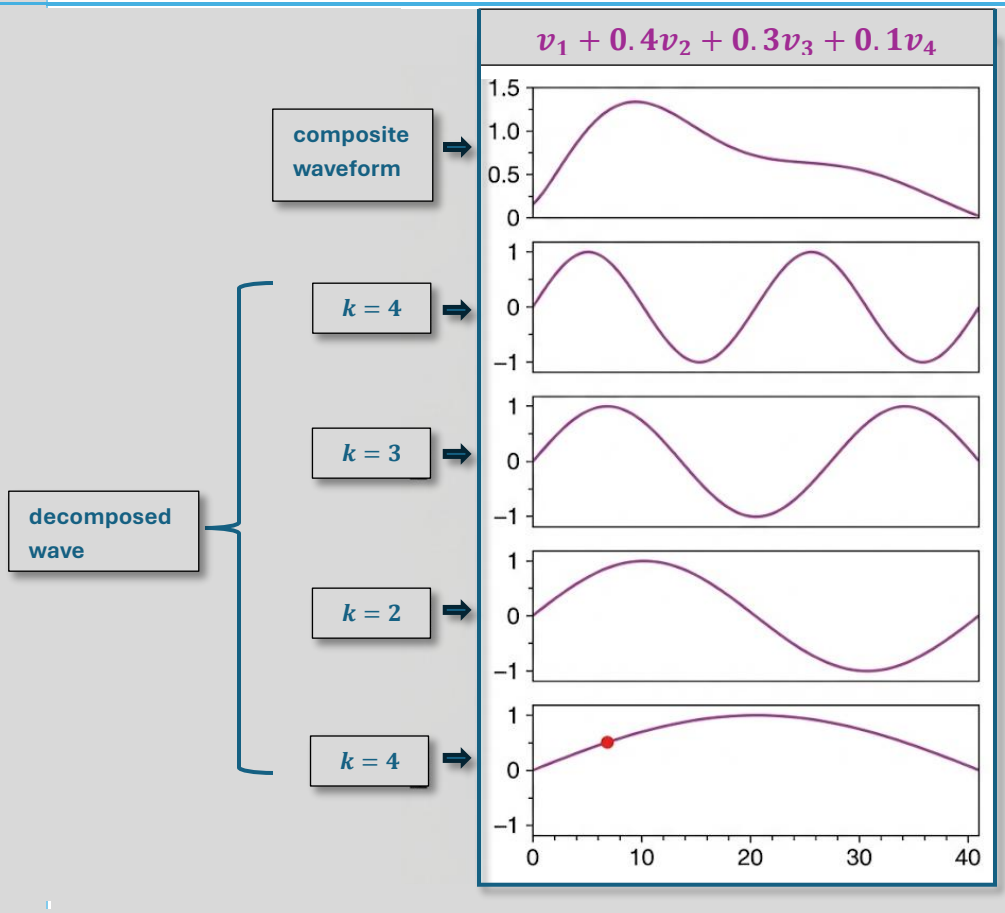
- $x = v_1 + 0.4v_2 + 0.3v_3 + 0.1v_4 = Vx = V$

$$\begin{bmatrix} 1 \\ 0.4 \\ 0.3 \\ 0.1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

- the ***sin* transform** of x is the **coordinate** with respect to the **basis**

Orthonormal: https://www.kristakingmath.com/blog/orthonormal-basis-for-a-vector-set

Visualization



DFT – Numerical View

Computing

- small variation in what numerical methods compute
- fundamentally there is a standard matrix V

$$V_{n,k} = \exp\left(i\frac{2\pi ikn}{N}\right) \qquad V^{-1} = \frac{1}{N}V^H$$

- there is flexibility where you put that N factor numerically

$$\boldsymbol{c = \frac{1}{N}V^H x} \quad \boldsymbol{x = V c}$$ ← easier to motivate

$$\boldsymbol{c = V^H x} \quad \boldsymbol{x = \frac{1}{N}V c}$$ ← FFTW

$$\boldsymbol{c = \frac{1}{\sqrt{N}}V^H x} \quad \boldsymbol{x = \frac{1}{\sqrt{N}}V c}$$ ← orthogonal (truly)

Fast Fourier Transform (FFT) in Julia (ex 1)

```
julia> using FFTW

julia> x = Vector(range(start=0, step=0.1, length=10))
10-element Vector{Float64}:
 0.0
 0.1
 0.2
 0.3
 0.4
 0.5
 0.6
 0.7
 0.8
 0.9

julia> y = sin.(2pi*x)
10-element Vector{ComplexF64}:
 1.2246467991473532e-16 + 0.0im
-5.50555994384384e-16 + 5.0im
 1.2246467991473532e-16 - 2.1117696842213397e-16im
 1.2246467991473532e-16 - 4.440892098500626e-16im
 1.2246467991473532e-16 - 1.305145441260248e-16im
 9.957992501029599e-17 + 0.0im
 1.2246467991473532e-16 + 1.305145441260248e-16im
 1.9440492184190732e-16 + 4.440892098500626e-16im
 1.2246467991473532e-16 + 2.1117696842213397e-16im
-5.50555994384384e-16 + 5.0im
```

$$x_n = \frac{n}{N}$$

$$y_n = sin\,(2\pi\frac{n}{N})$$

Fast Fourier Transform (FFT) in Julia (ex 2)

```
julia> using FFTW

julia> x = Vector(range(0, 1, 11))
11-element Vector{Float64}:
 0.0
 0.1
 0.2
 0.3
 0.4
 0.5
 0.6
 0.7
 0.8
 0.9
 1.0

julia> y = sin.(2pi*x)

julia> fft(y)
11-element Vector{ComplexF64}:
-1.1102230246251565e-16 + 0.0im
 4.447252663038674 + 4.928889952937341im
-0.4364947971936885 + 0.6791991628794149im
-0.35239245098458942 + 0.30578312732722llim
-0.3323496580110257 + 0.15172662498974 83im
-0.3256303599998888 + 0.046818577529318 11im
-0.3323496580110257 - 0.15172662498974 83im
-0.35239245098458942 - 0.30578312732722llim
-0.4364947971936885 - 0.6791991628794149im
 4.447252663038674 - 4.928889952937341im
-1.1102230246251565e-16 + 0.0im
```

Toeplitz Matrix

a **structured diagonal**-constant **matrix** where each **descending diagonal** from left to right **contains** the **same elements** defined $\boldsymbol{T_{(ij)} = t_{(i-j)}}$. these **matrices** are used in **signal** processing, **image** processing, and **time series analysis** for their **efficiency** in **solving** typically **requiring** only $\boldsymbol{O(n)}$ distinct **elements** and $\boldsymbol{O(n^2)}$to **solve** $\boldsymbol{Ax = y}$.